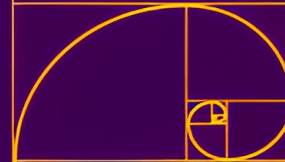


Function Analysis

CSCI 33500: Software Design and Analysis III

Sadab Hafiz
sh3646@hunter.cuny.edu

03



Functions

What are functions?

- A function (or subroutine) is code which has been separated out for:
 - ◆ Repeated set of operations
 - E.g., complex math functions
 - ◆ Grouping related tasks
 - E.g., initialization
- Because a function can be called from anywhere, we must:
 - ◆ Prepare the appropriate environment
 - ◆ Deal with the arguments (parameters)
 - ◆ Jump to the subroutine
 - ◆ Execute the subroutine
 - ◆ Deal with the return value
 - ◆ Clean up

Function Call Example

Jump to a Different Set of Instructions and Execute

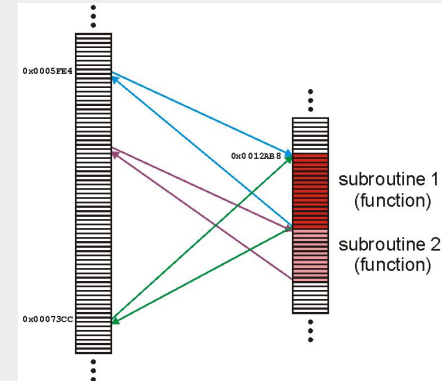
```
#include<iostream>

class Student {...};

Student createStudent() {
    Student student;
    ... ..
    return student;
}

int main() {
    ... ..
    student1 = createStudent();
    ... ..
}
```

function call



Functions

Complexity

- Fortunately, calling a function is such a common task that all modern processors have instructions that perform most of these steps in just one instruction
- Thus, we will assume that the overhead required to make a function call and return is $\Theta(1)$
- Because any function requires the overhead of a function call, we will always assume $T_f = \Omega(1)$
- That is, it is impossible for any function call to have zero runtime

Function Analysis Example

```
void Disjoint_sets::set_union( int m, int n ) {  
    m = find( m );  
    n = find( n );  
  
    if ( m == n ) {  
        return;  
    }  
  
    --num_disjoint_sets;  
  
    if ( tree_height[m] >= tree_height[n] ) {  
        parent[n] = m;  
  
        if ( tree_height[m] == tree_height[n] ) {  
            ++( tree_height[m] );  
            max_height = std::max( max_height, tree_height[m] );  
        }  
    } else {  
        parent[m] = n;  
    }  
}
```

$2T_{\text{find}}$

$T_{\text{set_union}} = 2T_{\text{find}} + \Theta(1)$

$\Theta(1)$

→ Given a function $f(n)$ (the runtime of which depends on n), we will associate the runtime of $f(n)$ by some function $T_f(n)$ or $T(n)$

→ Because the runtime of any function is at least $O(1)$, we will include the time required to both call and return from the function in the runtime

Recursive Functions

Function That Calls Itself

- A function is relatively simple (and boring) if it simply performs operations and calls other functions
- Most interesting functions that are designed to solve problems usually end up calling themselves, such functions are called **recursive functions**

Recursion Example

Factorial

```
int factorial ( int n ) {  
    if ( n == 0 ) {  
        return 1;  
    } else {  
        return n * factorial ( n - 1 );  
    }  
}
```

$$T_!(n) = \begin{cases} \Theta(1) & n \leq 1 \\ T_!(n - 1) + \Theta(1) & n > 1 \end{cases}$$

We don't have to worry about the time of the conditional($\Theta(1)$) nor is there a probability involved with the statement

Factorial

Recurrence

- The analysis of the runtime yields a recurrence relation:

$$T_i(n) = T_i(n - 1) + \Theta(1) \quad T_i(1) = \Theta(1)$$

- This recurrence relation has Landau symbols. We replace each with a representative function:

$$T_i(n) = T_i(n - 1) + 1 \quad T_i(1) = 1$$

Recurrence Relations

Recursion in Math

→ Sequences may be defined explicitly:
 $x_n = 1/n$

$$1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots$$

→ A recurrence relationship is a means of defining a sequence based on previous values in the sequence

Steps:

→ Define an initial value:

◆ E.g., $x_1 = 1$

→ Define x_n in terms of the previous values. For example:

◆ $x_n = x_{n-1} + 2$

◆ $x_n = 2x_{n-1} + n$

◆ $x_n = x_{n-1} + x_{n-2}$

Recurrence Notation

Calculus vs Functional Form

→ Calculus:

- ◆ $x_1 = 1$
- ◆ $x_n = x_{n-1} + 2$
- ◆ $x_n = 2x_{n-1} + n$

→ Functional form:

- ◆ $f(1) = 1$
- ◆ $f(n) = f(n-1) + 2$
- ◆ $f(n) = 2 f(n-1) + n$

→ We will use the functional form

Recurrence

Examples

→ Example 1:

- ◆ $f(1) = 1$
- ◆ $f(n) = f(n - 1) + 2$
- ◆ Sequence: 1, 3, 5, 7, 9, ...
- ◆ Solution: $f(n) = 2n - 1$
- ◆ The n th odd number

→ Example 2:

- ◆ $g(1) = 1$
- ◆ $g(n) = 2g(n - 1) + n$
- ◆ Sequence: 1, 4, 11, 26, 57, ...
- ◆ Solution: $g(n) = 2^{n+1} - n - 2$

→ Not all recurrence relations can be solved for explicit formulas

Factorial

Solving the Recurrence

- To find the run time of the factorial function, we need to solve:

$$T_!(n) = T_!(n - 1) + 1 \quad T_!(1) = 1$$

- Let's examine the first few steps:

$$\begin{aligned} T_!(n) &= T_!(n - 1) + 1 \\ &= T_!(n - 2) + 1 + 1 = T_!(n - 2) + 2 \\ &= T_!(n - 3) + 3 \end{aligned}$$

- From this, we can see a pattern:

$$T_!(n) = T_!(n - k) + k$$

Factorial

Solving the Recurrence

```
int factorial ( int n ) {  
    if ( n == 0 ) {  
        return 1;  
    } else {  
        return n * factorial ( n - 1 );  
    }  
}
```

→ Pattern: $T_1(n) = T_1(n - k) + k$

→ If $k = n - 1$, then:

$$\begin{aligned} T_1(n) &= T_1(n - (n - 1)) + (n - 1) \\ &= T_1(1) + n - 1 \\ &= 1 + n - 1 \\ &= n \end{aligned}$$

→ Thus, $T_1(n) = \Theta(n)$

Ternary Operator

?

- You can also write the factorial function using the ternary ?
- ◆ Only C++ operator with three arguments
 - ◆ (condition) ? (true code) : (false code)
 - ◆ Behaves similarly to if-else statement
 - ◆ No benefits in terms of performance
 - ◆ Useful for readability and ease

```
int factorial( int n ) {  
    return (n <= 1) ? 1 : n * factorial( n - 1 );  
}
```

Recursive Selection Sort

Another Example

- To sort an array of n items, We can use Selection Sort
- Previously we have seen an iterative implementation. It can also be implemented recursively:
 1. Go through the array and find the largest item
 2. Swap the last entry with the largest item
 3. Recursive call to sort the rest of the array

Recursive Selection Sort Code

```
void sort( int * array, int n ) {  
    if ( n <= 1 ) {  
        return;                // special case: 0 or 1 items are always sorted  
    }  
  
    int posn = 0;                // assume the first entry is the smallest  
    int max = array[posn];  
  
    for ( int i = 1; i < n; ++i ) { // search through the remaining entries  
        if ( array[i] > max ) { // if a larger one is found  
            posn = i;           // update both the position and value  
            max = array[posn];  
        }  
    }  
  
    int tmp = array[n - 1];        // swap the largest entry with the last  
    array[n - 1] = array[posn];  
    array[posn] = tmp;  
  
    sort( array, n - 1 );          // sort everything else  
}
```


Trace Example

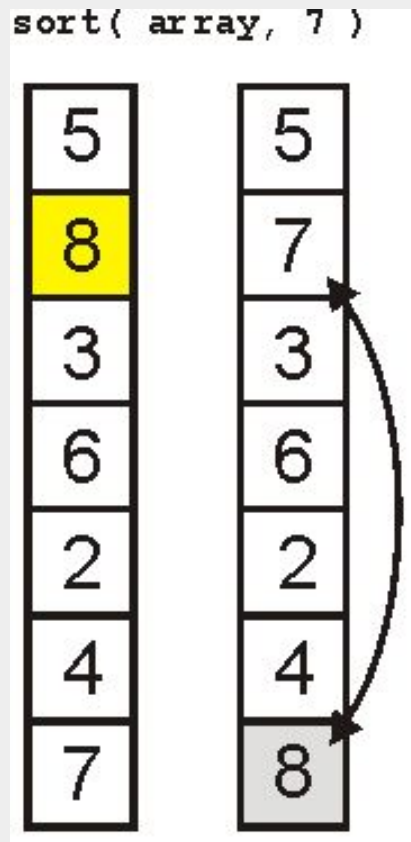
Recursive Selection Sort

```
int array[7] = {5, 8, 3, 6, 2, 4, 7};  
sort( array, 7 ); // array of 7 items
```

array	
5	
8	
3	
6	
2	
4	
7	

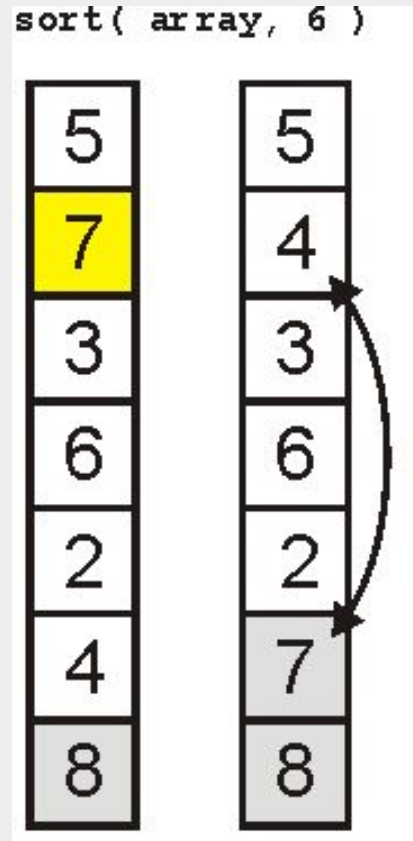
First Call

Finds The Largest Element



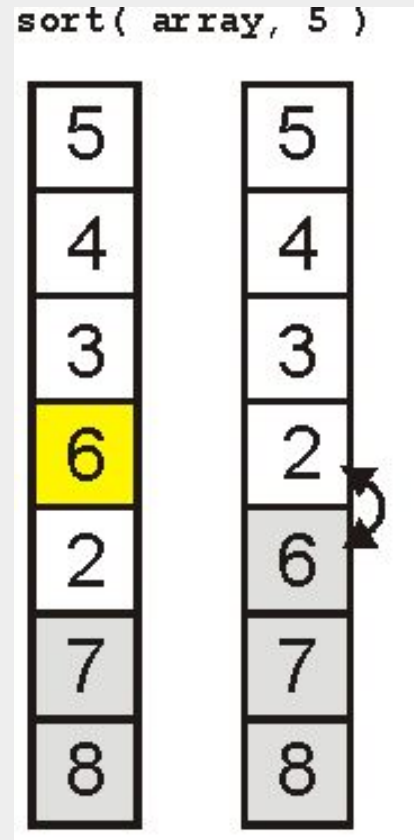
2nd Call

Finds The 2nd Largest Element



3rd Call

Finds The 3rd Largest Element



Sorting the Rest of the Array

`sort( array, 4 )`



`sort( array, 3 )`



`sort( array, 2 )`



`sort( array, 1 )`



Recursive Selection Sort Runtime

```
void sort( int * array, int n ) {  
    if ( n <= 1 ) {  
        return;  
    }  
  
    int posn = 0;  
    int max = array[posn];  
  
    for ( int i = 1; i < n; ++i ) {  
        if ( array[i] > max ) {  
            posn = i;  
            max = array[posn];  
        }  
    }  
  
    int tmp = array[n - 1];  
    array[n - 1] = array[posn];  
    array[posn] = tmp;  
  
    sort( array, n - 1 );  
}
```

Annotations for runtime analysis:

- $T(0) = T(1) = \Theta(1)$ (for the base case)
- $\Theta(1)$ (for finding the maximum element)
- $\Theta(1)$ (for the inner loop body)
- $\Theta(n)$ (for the outer loop)
- $\Theta(1)$ (for the swap operation)
- $T(n-1)$ (for the recursive call)

Recurrence relation:

$$T(n) = \Theta(1) + \Theta(n) + \Theta(1) + T(n-1)$$
$$= T(n-1) + \Theta(n)$$

Selection Sort Analysis

Recurrence

Is this better than the iterative approach we saw in class?

→ Recurrence:

$$T(n) = T(n - 1) + \Theta(n) \quad T(1) = 1$$

→ Replace Landau symbols:

$$T(n) = T(n - 1) + n \quad T(1) = 1$$

→ Solving this leads to:

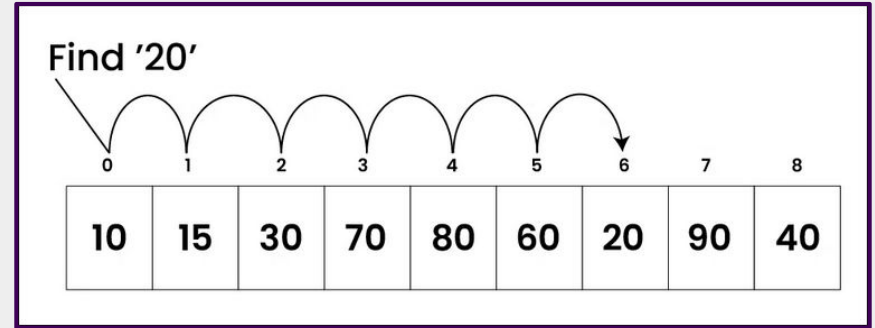
$$\begin{aligned} T(n) &= T(n - 1) + n \\ &= (T(n - 2) + (n - 1)) + n \\ &= T(n - 2) + n + (n - 1) \\ &= T(n - 3) + n + (n - 1) + (n - 2) \\ &\vdots \\ &= T(1) + \sum_{i=2}^n i = 1 + \sum_{i=2}^n i = \sum_{i=1}^n i = \frac{n(n+1)}{2} \end{aligned}$$

→ Thus, the runtime is $\Theta(n^2)$

Linear Search

Searching a List

What is the worst case scenario?



- Given any arbitrary list of items, the goal is to find the index of item x
- We must check each entry one by one iterating the entire list
- Can exit early if the item is found

Binary Search

Recursive Approach

Can you implement this?

- Linear search is $O(n)$
- We can do better than linear search by using binary search **given that the list is sorted**
- Binary search is a divide and conquer strategy:
 - ◆ Suppose, we are trying to find entry x
 - ◆ Check the middle entry for x
 - ◆ If we don't find it:
 - Compare middle entry to x
 - If less, search the right half
 - If greater, search the left half

Binary Search Analysis

Recurrence

→ Binary search recurrence:

- ◆ $(n - 1) / 2$ is the size of the sub array

$$T(n) = T((n - 1)/2) + \Theta(1)$$

→ Also, if $n = 1$, then $T(1) = \Theta(1)$

→ Thus, we have to solve:

$$T(n) = \begin{cases} 1 & n = 1 \\ T\left(\frac{n - 1}{2}\right) + 1 & n > 1 \end{cases}$$

Binary Search Analysis

Recurrence

$$T(n) = \begin{cases} 1 & n = 1 \\ T\left(\frac{n-1}{2}\right) + 1 & n > 1 \end{cases}$$

- Solving this can be difficult, because $n > 1$ is a large range, so we will only consider special values of n
- Assume $n = 2^k - 1$
 - ◆ where k is an integer
- Then:
 - ◆ $(n - 1) / 2 = (2^k - 1 - 1) / 2 = 2^{k-1} - 1$

Binary Search Analysis

Recurrence

$$T(n) = \begin{cases} 1 & n = 1 \\ T\left(\frac{n-1}{2}\right) + 1 & n > 1 \end{cases}$$

- Solving this can be difficult, because $n > 1$ is a large range. We will only consider special values of n .
- Assume $n = 2^k - 1$
 - ◆ where k is an integer
- Then:
 - ◆ $(n - 1) / 2 = (2^k - 1 - 1) / 2 = 2^{k-1} - 1$
- For example, if we are checking a list of size 31 and the item is not in the middle, we must check one of the two halves (size 15):
 - ◆ $31 = 2^5 - 1$
 - ◆ $15 = 2^4 - 1$

Binary Search Analysis

Recurrence

$$T(n) = \begin{cases} 1 & n = 1 \\ T\left(\frac{n-1}{2}\right) + 1 & n > 1 \end{cases}$$

→ Thus, we can write:

$$\begin{aligned} T(n) &= T(2^k - 1) \\ &= T\left(\frac{2^k - 1 - 1}{2}\right) + 1 \\ &= T(2^{k-1} - 1) + 1 \\ &= T\left(\frac{2^{k-1} - 1 - 1}{2}\right) + 1 + 1 \\ &= T(2^{k-2} - 1) + 2 \\ &\vdots \end{aligned}$$

Binary Search Analysis

Recurrence

$$T(n) = \begin{cases} 1 & n = 1 \\ T\left(\frac{n-1}{2}\right) + 1 & n > 1 \end{cases}$$

→ Notice the pattern:

$$\begin{aligned} &= T(2^{k-1} - 1) + 1 \\ &= T\left(\frac{2^{k-1} - 1 - 1}{2}\right) + 1 + 1 \\ &= T(2^{k-2} - 1) + 2 \\ &= T(2^{k-3} - 1) + 3 \\ &\vdots \end{aligned}$$

Binary Search Analysis

Recurrence

$$T(n) = \begin{cases} 1 & n = 1 \\ T\left(\frac{n-1}{2}\right) + 1 & n > 1 \end{cases}$$

→ Thus, in general, we may deduce that after $k - 1$ steps:

$$\begin{aligned} T(n) &= T(2^k - 1) \\ &= T(2^{k-(k-1)} - 1) + k - 1 \\ &= T(1) + k - 1 = k \end{aligned}$$

because $T(1) = 1$

→ Thus, $T(n) = k$, but $n = 2^k - 1$

→ Solving for k , $k = \log_2(n + 1)$

Binary Search Analysis (Θ Limit Test)

→ To check if this is a good estimate for Θ , recall:

$$f(n) = \Theta(g(n)) \text{ if } \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c \text{ for } 0 < c < \infty$$

$$\lim_{n \rightarrow \infty} \frac{\lg(n+1)}{\ln(n)} = \lim_{n \rightarrow \infty} \frac{\frac{1}{(n+1)\ln(2)}}{\frac{1}{n}} = \lim_{n \rightarrow \infty} \frac{n}{(n+1)\ln(2)} = \frac{1}{\ln(2)}$$

→ Thus, $T(n) = \Theta(\log_2(n+1)) = \Theta(\log_2(n))$

When to Use Recursion?

```
int factorial_tail( int n, int result ) {  
    if ( n <= 1 ) {  
        return result;  
    } else {  
        return factorial_tail( n - 1, n * result );  
    }  
}
```

- Elegant alternative to highly complex iterative logic
 - ◆ Implementing Divide and Conquer
 - E.g., Merge Sort, Quick Sort, etc.
- Some data structures are recursive
 - ◆ E.g., Binary Search Trees
 - ◆ Easier to implement recursive traversal
- Compilers can optimize tail recursive functions to be iterative
 - ◆ Using [Tail Call Optimization \(TCO\)](#)
 - Recursive call is the last statement
 - ◆ No memory overhead and stack usage
 - ◆ Supported by some languages
 - C, C++, Scala, etc.
 - ◆ Not supported by some languages
 - Python, Java, etc.